SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type : Lab**	**Academic Year :** 2025-2026
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week3 – Monday**	**Batch**	23CSBTB47B
Name	Aelishala Manoj	**Hall Ticket No**	2303A540
Assignment Number: 5.1			

Q.No.	Question
	Lab 5: Ethical Foundations – Responsible AI Coding Practices **Lab Objectives:** • To explore the ethical risks associated with AI-generated code. • To recognize issues related to security, bias, transparency, and copyright. • To reflect on the responsibilities of developers when using AI tools in software development. • To promote awareness of best practices for responsible and ethical AI coding. **Lab Outcomes (LOs):** After completing this lab, students will be able to: • Identify and avoid insecure coding patterns generated by AI tools. • Detect and analyze potential bias or discriminatory logic in AI-generated outputs. • Evaluate originality and licensing concerns in reused AI-generated code. • Understand the importance of explainability and transparency in AI-assisted programming. • Reflect on accountability and the human role in ethical AI coding practices. **Task Description #1 (Privacy in API Usage)** Task: Use an AI tool to generate a Python program that connects to a weather API. Prompt: *"Generate code to fetch weather data securely without exposing API keys in the code."* Expected Output: • Original AI code (check if keys are hardcoded). • Secure version using environment variables.

lab5.1-4054.py .env

lab5.1-4054.py > ...

```python
import requests

CITY = "London"
LAT = 51.5072
LON = -0.1276

URL = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": LAT,
    "longitude": LON,
    "current_weather": True
}

response = requests.get(URL, params=params)

if response.status_code == 200:
    data = response.json()
    weather = data["current_weather"]
    print({
        "city": CITY,
        "temperature": f"{weather['temperature']}°C",
        "windspeed": weather["windspeed"]
    })
else:
    print("API error")
```

Output:

```
/Desktop/AI-AS/lab5.1-4054.py
{'city': 'London', 'temperature': '7.1°C', 'windspeed': 9.8}
```

Task Description #2 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file.

Analyze: Check if the AI stores sensitive data in plain text or without encryption.

Expected Output:

- Identified privacy risks.
- Revised version with encrypted password storage (e.g., hashing).

Prompt :

Generate a Python script to store user data (name, email, password) in a JSON file. NEVER store passwords in plaintext use PBK model with a unique salt (os.urandom(32)) and 100k iterations. Include verify function. Add comments on why plaintext is insecure.

Code :

lab5.1-4054.py

```python
# Task 2
import hashlib
import os
import json

def store_user_data(name, email, password):
    # Generate unique salt per user
    salt = os.urandom(32)
    # Hash password with PBKDF2 (industry standard, slow against brute-force)
    hashed_pw = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    user_data = {
        'name': name,
        'email': email,
        'salt': salt.hex(),
        'hashed_password': hashed_pw.hex()
    }
    with open('users.json', 'w') as f:
        json.dump(user_data, f)  # JSON for structured, parseable storage
    print("User data stored securely.")

# Demo usage (don't hardcode in prod)
store_user_data('John Doe', 'john@email.com', 'secret123')

```

Output:

```
/Desktop/AI-AS/lab5.1-4054.py
User data stored securely.
```

Explanation:

AI saves name, email, password straight to file readable by anyone, total hack risk.

Fix: Mash password into unreadable hash + random salt; store that instead can't steal real password.

Task Description #3 (Transparency in Algorithm Design)

Objective: Use AI to generate an Armstrong number checking function with comments and explanations.

Instructions:

1. Ask AI to explain the code line-by-line.
2. Compare the explanation with code functionality.

Expected Output:

- Transparent, commented code.
- Correct, easy-to-understand explanation.

Prompt :

Write Python function to check Armstrong number (sum of digits^digit-count equals number, e.g. 153). Add line-by-line comments, handle negatives/zero, test 153/407/100.

Code :

lab5.1-4054.py

lab5.1-4054.py > is_armstrong

```
56  # # Task 3
57  def is_armstrong(num):
58      """
59      Checks if a number is Armstrong: sum of its digits^num_digits equals num.
60      E.g., 153 = 1^3 + 5^3 + 3^3.
61      """
62      if num < 0:
63          return False
64      original = num
65      num_digits = len(str(num))
66      total = 0
67      while num > 0:
68          digit = num % 10
69          total += digit ** num_digits  # Power each digit
70          num //= 10  # Drop last digit
71      return total == original
72
73  # Test
74  print(is_armstrong(153))  # True
75  print(is_armstrong(407))  # True
76  print(is_armstrong(100))  # False
77
78  # # End of Task 3
```

Output :

```
/Desktop/AI-AS/lab5.1-4054.py
True
True
False
```

Explanation:

Armstrong number: A number equals the sum of its digits each raised to the power of total digit count.

Code grabs each digit, powers it correctly, adds them up, then checks if that matches the original.

Task Description #4 (Transparency in Algorithm Comparison)

Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt:

"Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ."

Expected Output:

- Code for both algorithms.
- Transparent, comparative explanation of their logic and efficiency.

Codes :

lab5.1-4054.py

```python
# # Task 4

#Quick Sort Implementation
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
# Test
print(quick_sort([3,6,8,10,1,2,1]))

# Bubble Sort Implementation
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Test
print(bubble_sort([3,6,8,10,1,2,1]))
```

Output :

```
/Desktop/AI-AS/lab5.1-4054.py
[1, 1, 2, 3, 6, 8, 10]
[11, 12, 22, 25, 34, 64, 90]
PS C:\Users\alish\OneDrive\Docu
/Desktop/AI-AS/lab5.1-4054.py
[1, 1, 2, 3, 6, 8, 10]
[1, 1, 2, 3, 6, 8, 10]
```

Explanation:

BubbleSort slowly swaps neighbor pairs until none needed simple but crawls on big lists.

QuickSort picks middle value, splits smaller/larger sides, repeats divide fast splitter for real work.

Task Description #5 (Transparency in AI Recommendations)

Task: Use AI to create a product recommendation system.

Prompt:

"Generate a recommendation system that also provides reasons for each suggestion."

Expected Output:

- Code with explainable recommendations.
- Evaluation of whether explanations are understandable.

Code :

lab5.1-4054.py

```
# # Task 5
ratings = {
    'Alice': {'laptop': 5, 'phone': 3, 'headphones': 4, 'watch': 2},
    'Bob': {'laptop': 4, 'phone': 5, 'shoes': 4},
    'Charlie': {'headphones': 5, 'shoes': 3, 'watch': 4}
}

def recommend(user, ratings):
    user_items = ratings[user]
    suggestions = []
    for other, other_items in ratings.items():
        if other == user: continue
        sim_items = set(user_items) & set(other_items)
        if sim_items:
            sim_score = sum(user_items[item] * other_items[item] for item in sim_items) / len(sim_items)
            for item, score in other_items.items():
                if item not in user_items and score >= 4:
                    reason = f"Bob (similar on laptop/phone) loved it ({score}/5), taste match {sim_score:.1f}"
                    suggestions.append((item, reason))
    return sorted(set(suggestions), key=lambda x: x[1])[:3]  # Unique top 3

print(recommend('Alice', ratings))
# [('shoes', 'Bob (similar on laptop/phone) loved it (4/5), taste match 4.0'), ('headphones', ...)]

```

Output:

```
/Desktop/AI-AS/lab5.1-4054.py
[('shoes', 'Bob (similar on laptop/phone) loved it (4/5), taste match 17.5')]
```

Explanation:

Recommendation system scans other users' tastes matching yours, then suggests their high rated items you lack.

Reasons spell out exactly who liked what and why tastes align clear path from data to suggestion, no magic.